UNIVERSITÀ DI PISA

Computer Engineering

882II Internet of Things

Project Documentation

Demand Side Management (DSM) Application

Author:
Sfar Omar Tomàs

Academic Year: 2025-2026

Contents

1 Introduction

The Internet of Things (IoT) is changing how we manage energy in smart buildings. Demand Side Management (DSM) continuously monitors and controls energy use to improve efficiency and prevent power grid overloads.

Modern DSM systems use Edge Computing. By running Machine Learning (ML) models directly on edge devices, IoT sensors can predict energy spikes and make fast decisions. This saves network bandwidth and allows safety actions, like turning off loads, even without an internet connection.

This project builds a resilient DSM application. It integrates Edge Machine Learning, device-to-device communication, and Cloud control. The system implements some adaptive mechanisms to survive network failures and hardware faults automatically.

1.1 Use-Case Scenario: Smart Building Energy Management

The specific use-case implemented in this project is a **Demand Side Management (DSM) system for a Smart Building**.

In a modern facility, the simultaneous activation of heavy electrical loads (e.g., HVAC systems, industrial machinery, or EV chargers) can lead to sudden power peaks. If these peaks exceed the local grid's capacity, they can cause dangerous blackouts or incur severe economic penalties from the energy provider.

To prevent this, the building is equipped with IoT smart meters (Edge nodes) attached to critical appliances. The system operates on two distinct levels:

- **Micro-level (Edge):** If a single appliance suddenly malfunctions or draws an anomalous amount of power (e.g., local peak > 5.0 kW), the attached sensor immediately predicts the anomaly via Machine Learning, cuts the power to isolate the fault, engages a physical safety lock (Red LED), and alerts nearby nodes.
- **Macro-level (Cloud):** A central Cloud application continuously monitors the aggregate power consumption of the entire building. If the global average exceeds a safe baseline threshold (e.g., > 4.0 kW), it automatically broadcasts a CoAP command to turn off non-essential loads, keeping the building grid stable.

This dual-layer approach guarantees both local safety against short-circuits and global stability against grid overloads.

2 Project Requirements

The project guidelines are the followings:

- **REQ-01 (IoT Network):** The system MUST comprise a network of IoT devices, including sensors simulating the collection of data from the physical system/environment and actuators.
- **REQ-02 (Deployment & Border Router):** The network MUST be deployed using real sensors (nrf52840 dongle). A Border Router MUST be deployed in order to provide external IPv6 access.

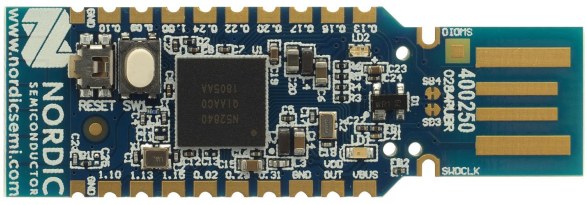

Figure 1: nrf52840 dongle (front)

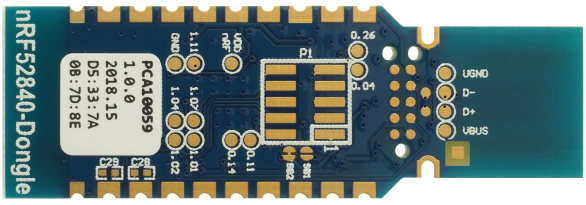

Figure 2: nrf52840 dongle (back)

- **REQ-03 (Application Protocol):** The application-layer protocol adopted by the IoT devices MUST be CoAP. Devices using CoAP MUST implement both CoAP Server functionality to expose their resources, and CoAP Client functionality for interacting with other devices.
- **REQ-04 (Edge Machine Learning):** A machine learning model MUST run on the IoT devices to take autonomous decisions directly on the Edge (e.g., forecasting metrics and reacting to anomalies). The model MUST be trained using an open dataset.
- **REQ-05 (Cloud Application & DB):** The Cloud Application MUST collect data from CoAP sensors and store them in a database.
- **REQ-06 (Closed-Loop Control):** The Cloud Application MUST read information from the database and implement a simple control logic to apply modifications to one or more actuators based on the collected data.
- **REQ-07 (User Input):** The system MUST provide a Command Line Interface (CLI) to implement the User logic for the IoT application.
- **REQ-08 (Device Directory):** The list of actuators and sensors MUST be statically created and exploited by the User Application via a configuration file.
- **REQ-09 (Stress Testing):** The system MUST be experimentally evaluated under a *Node Failure / Intermittent Behavior Scenario*. The evaluation must show the system's ability to detect faulty nodes, dropping messages, or inconsistent data.
- **REQ-10 (Adaptive Mechanism):** The system MUST include at least one adaptive mechanism that reacts to the stress conditions (e.g., adaptation of control logic based on system conditions).
- **REQ-11 (Web Dashboard):** A web-based interface deployed using Grafana MUST be developed to show the performance metrics of the stress tests.

3 Tools and Technologies

To design, simulate, and implement the system, we used a specific set of software tools:

- **Contiki-NG:** Used for the Edge layer. Contiki-NG is the operating system for the IoT nodes, written in C.
- **Python and Jupyter Notebook:** Used to analyze the public energy dataset, train the Machine Learning model, and extract its parameters to use them in the C code.
- **Java and Californium Framework:** Used to build the Cloud Application. Californium is a Java library that provides the engine to send and receive CoAP messages.
- **InfluxDB:** A time-series database used to store all the telemetry data.
- **Grafana:** A web application connected to InfluxDB to visualize the data and the stress tests in real-time.

4 System Architecture

The DSM application uses a multi-tier architecture, from Edge devices to Cloud application.

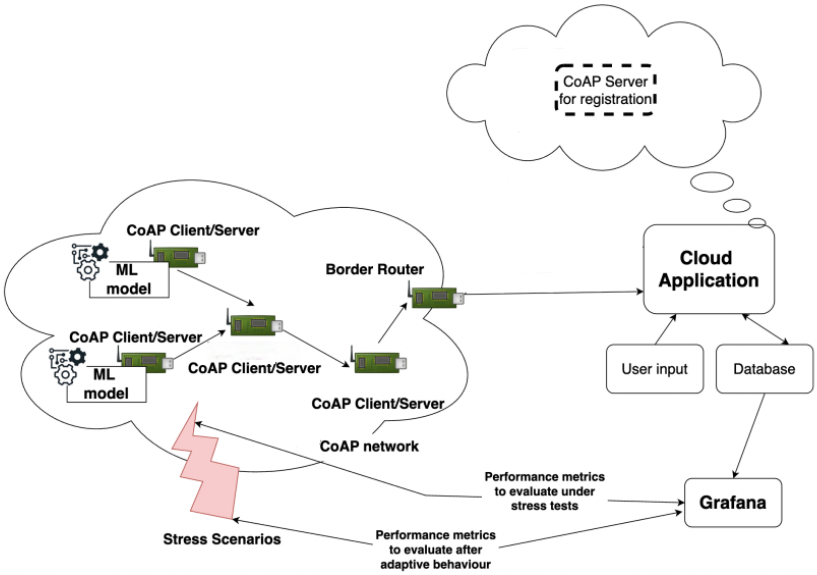

Figure 3: System block diagram

4.1 Protocol Selection: CoAP vs. MQTT

We chose CoAP over MQTT to satisfy **REQ-03** for two main reasons:

- **Decentralized Communication:** Our system needs fast, local responses. CoAP allows nodes to talk directly to each other (Peer-to-Peer). When a sensor predicts a power spike, it sends a command directly to a neighbor to turn off its load. MQTT needs a central broker, which adds delay and a single point of failure.
- **Transport Layer and Observe:** CoAP operates over UDP, reducing the connection overhead compared to MQTT (which relies on TCP).

4.2 Data Encoding Selection

We chose **JSON** to send data from the Edge to the Cloud because:

- **XML (Rejected):** It is too heavy. The tags waste bandwidth and battery.
- **Binary Encoding (Rejected):** It compresses well but needs complex libraries that use too much memory. Sending only two numbers does not justify this complexity.
- **JSON (Adopted):** It is the perfect balance. It is light, easy to read, and we can generate it easily in C without extra libraries.

5 Edge Implementation

The Edge layer consists of autonomous smart meters. They read power consumption, predict future trends using Machine Learning, and take immediate actions.

5.1 CoAP Resources

Each sensor provides two main CoAP resources:

- `/load`: An actuator resource. It accepts `PUT` requests (`mode=on` or `mode=off`) to turn the local power load on or off.
- `/power`: An observable sensor resource. It provides the real and predicted power formatted in JSON.

JSON Payload Generation

```
int length = snprintf((char *)buffer, preferred_size,
"{\"P\":%d.%02d,\"Pr\":%d.%02d}", p_int, p_frac, pr_int, pr_frac);
coap_set_header_content_format(response, APPLICATION_JSON);
```

5.2 Local Manual Control (Hardware Button)

In addition to network commands, the system supports direct physical interaction. The nRF52840 dongle features a built-in hardware button. We programmed a hardware interrupt so that whenever a user presses this button, the device immediately toggles its local electrical load (switching it from ON to OFF, or from OFF to ON). This provides a fast and reliable manual override without requiring any network connection.

5.3 Machine Learning Model at the Edge

To satisfy **REQ-04**, we trained a Machine Learning model using Python and TensorFlow on a public Household Power Consumption dataset. The model is a lightweight Artificial Neural Network (ANN) optimized for microcontrollers.

The development and execution follow these key steps:

- **Data Preparation (Sliding Window):** The model relies on a temporal sequence. It takes a sliding window of the last 5 power readings as input to predict the future 6th value.
- **Feature Normalization:** To help the neural network process the numbers correctly, the data is normalized using a `StandardScaler`. The calculated mean and scale factors are extracted from Python and hardcoded into the C firmware to normalize live data on the sensor.
- **Network Architecture:** The model consists of two hidden *Dense* layers (16 neurons each with ReLU activation) and one output neuron. The model proved to be highly accurate, achieving a Mean Absolute Error (MAE) of only 0.16 kW on the test set.
- **Edge Conversion:** After training, the TensorFlow model was converted into raw C code using the `emlearn` library. The resulting `power_predictor.h` file is extremely optimized: its size is only 5.8 KB. This occupies just 2.2% of the 256 KB RAM available on the nRF52840 nodes, proving it is perfectly suited for constrained Edge devices.

During execution, periodically, the IoT node adds the new reading to its memory window, normalizes the array, and runs the generated `emlearn` function to forecast the future power consumption autonomously.

5.4 Decentralized In-Network Control (M2M) and Safety Logic

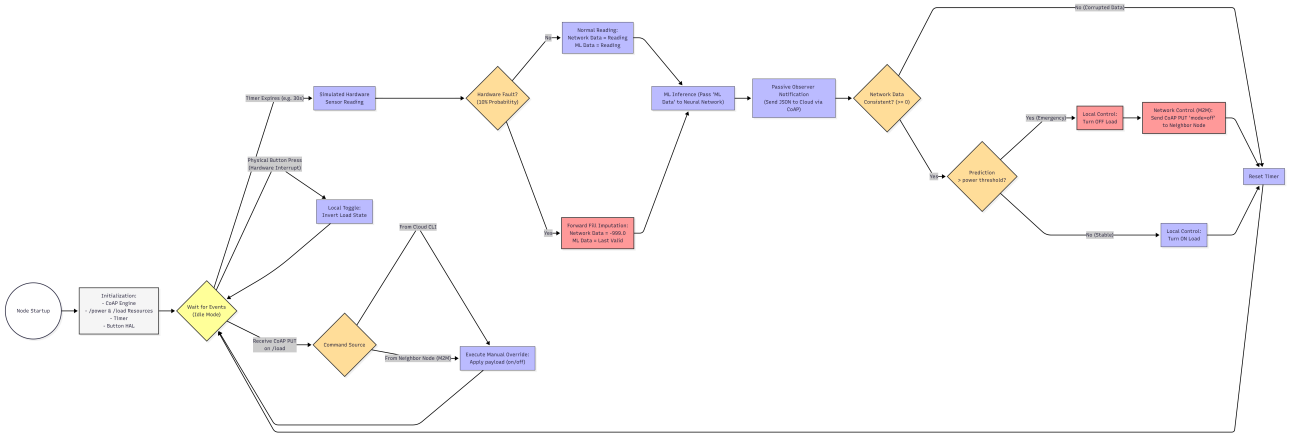

Figure 4: Block diagram of the sensor's life cycle

The Edge nodes operate with an asymmetric *Fail-Safe* logic. If the ML model predicts a local extreme power spike greater than the local threshold, the node reacts immediately:

1. **Local Action:** It forcefully turns off its own load and turns on a **Red Warning LED**.
2. **Network Action (M2M):** It acts as a CoAP Client and sends a `PUT` request directly to a neighbor to turn off its load too, bypassing the Cloud for instant reaction.

Note: The edge node is strictly restricted to turning the load OFF in emergencies. It cannot autonomously turn it back ON when the power drops. The restoration of power is delegated entirely to the Cloud Application.

5.5 Edge Resilience and Adaptive Mechanism

To fulfill the stress testing requirements (**REQ-09** and **REQ-10**), we added an adaptive defense mechanism.

During execution, the sensor has a 10% chance to fail and generate a bad reading (−999.0 kW). If this bad value enters the ML model, it ruins the predictions for the next 5 minutes.

To fix this, we implemented the *Forward Fill Imputation*. When a fault occurs, the node sends the −999.0 value to the Cloud to trigger alarms, but internally it uses the `last_valid_power` for the Machine Learning model.

Forward Fill Imputation (server.c)

```
if((random_rand() % 100) < 10) {
    current_power = -999.0f; // Bad data sent to Cloud
    ml_input = last_valid_power; // Forward Fill: Protects the ML
} else {
    current_power = sensor_reading;
    ml_input = sensor_reading;
    last_valid_power = sensor_reading; // Save safe value
}
predicted_power = run_ml_inference(ml_input);
```

This keeps the model mathematically stable.

6 Cloud Application and Control Logic

The Cloud Application is the central brain of the DSM system. It collects data, saves it, and controls the grid.

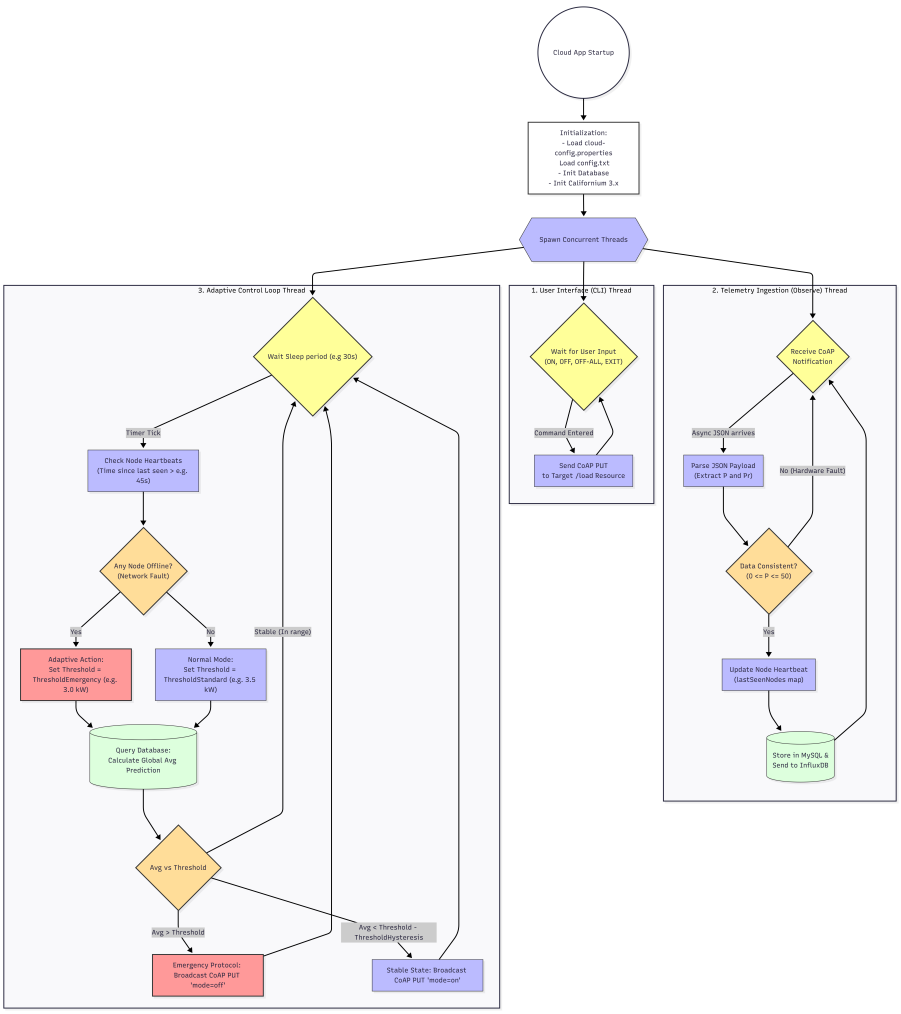

Figure 5: Block diagram of the Cloud's life cycle

6.1 Asynchronous Ingestion and Database

At startup, the application reads the node's IP list from `config.txt` (**REQ-08**), for simplicity we used only 2 nodes. It creates a CoAP client for each node and starts an *Observe* relationship.

When data arrives, the application parses the JSON payload to extract the real power (`P`) and the predicted power (`Pr`). Then, it forwards the data to the local Influx database using HTTP requests.

6.2 Core Control Loop and User Interaction

The Cloud application runs two concurrent tasks:

- **Closed-Loop Control (REQ-06):** Every 30 seconds, a background loop queries the database to calculate the global average predicted power. If the average exceeds the safety threshold, the Cloud sends a broadcast CoAP command (`mode=off`) to all nodes to protect the grid.
- **User Interface (CLI - REQ-07):** A separate thread provides a Command Line Interface. The user can type commands (`ON <ip>`, `OFF <ip>`, `OFF-ALL`) to manually override any actuator in the network.

6.3 System Control Logic (Finite State Machine)

To prevent race conditions between the Cloud macro-management and the Edge emergency reactions, the entire system follows a rigid Finite State Machine (FSM) architecture, enforcing a strict *Safety Interlock*.

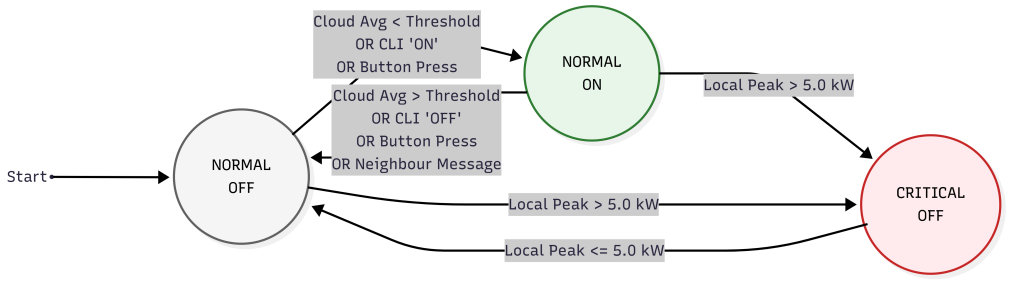

Figure 6: Finite State Machine representing the Control Logic and Safety Interlock.

The system operates across three distinct states:

- **NORMAL_OFF (No LED):** The safe default state. The electrical load is disconnected.
- **NORMAL_ON (Green LED):** The operational state. It is dynamically controlled by the Cloud (when average power drops below the threshold) or by user overrides.
- **CRITICAL_OFF (Red LED):** The emergency state, triggered exclusively by the local Edge node when its prediction exceeds the 5.0 kW limit.

As illustrated in the diagram, once a node enters `CRITICAL_OFF`, a safety interlock is engaged. All subsequent `ON` commands coming from the Cloud, the CLI, or the physical button are strictly ignored until the local anomaly resolves and the node returns to `NORMAL_OFF`.

7 Experimental Evaluation and Stress Testing

We tested the system in the simulator to see how it handles hardware and network faults (**REQ-09**). We monitored the results on a web dashboard (**REQ-11**).

7.1 Stress Scenario 1: Hardware Fault and Inconsistent Data

This scenario tests the system when a sensor generates bad data (-999.0 kW) randomly.

Without the Forward Fill mechanism, the bad value enters the ML model, causing the normalization to fail. The model breaks and generates absurd predictions (Figure 7), making the node blind to real overloads.

When the Forward Fill is active (**REQ-10**), the node protects its ML calculations. As shown in Figure 8, after an initial 5-minute phase to fill the memory window, the predictions track the real power with good precision, completely ignoring the faults.

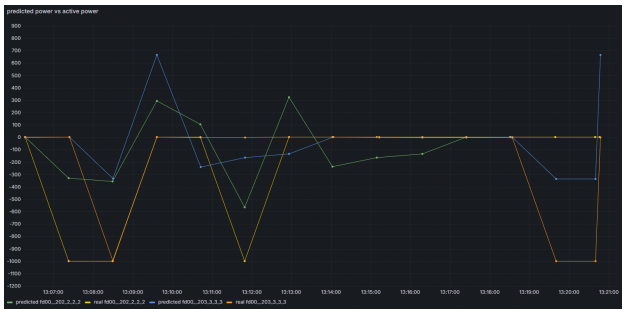

Figure 7: WITHOUT adaptation: predictions become absurdly out of scale.

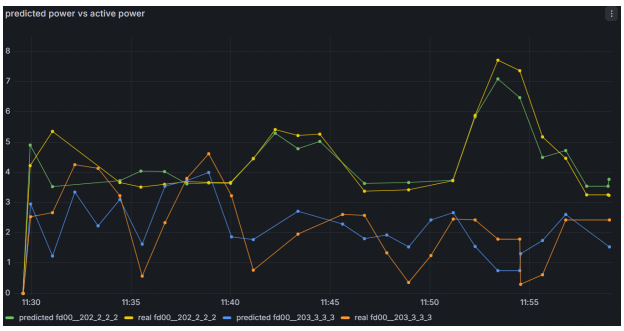

Figure 8: WITH adaptation: predictions accurately track real consumption.

7.2 Stress Scenario 2: Node Failure and Communication Loss

This scenario tests what happens if a node suddenly disconnects from the network, or if the transmission rate gets too low. Without adaptation, the Cloud keeps its safety threshold at a certain value. Since one node is missing, the Cloud calculates the average using only the remaining node. It becomes blind to a part of the building. The system fails to trigger the load reduction in time, causing a grid overload. With the *Dynamic Thresholding* active, the Cloud checks the heartbeat of all nodes. If a node is missing for more than the timeout (e.g. 45s), the Cloud detects the problem.

- Figure 9 shows the system detecting the failure, switching continuously between 2 and 1 active nodes. This behavior occurs because the intermittent node went offline for a short interval that did not trigger a total timeout of the CoAP Observe relationship; therefore, the Cloud automatically re-registered the heartbeat as soon as the node re-established communication.
- As a reaction, Figure 10 shows the Cloud dynamically changing the safety threshold.

By lowering the threshold, the system becomes more aggressive in turning off loads. This "Safe Mode" guarantees the grid safety even if the network is partially blind.

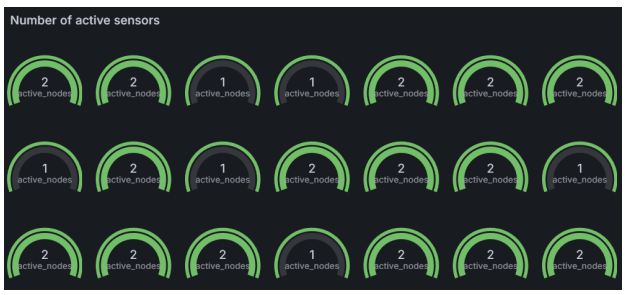

Figure 9: Active nodes counter dropping whenever there's a failure.

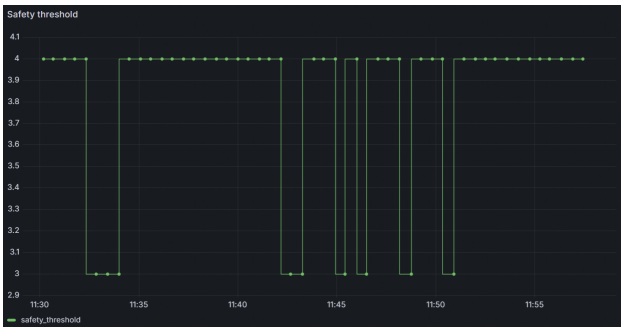

Figure 10: Safety threshold dynamically changing.

8 Conclusions

This project successfully built a DSM IoT application that meets all the requirements.

By moving Machine Learning to the Edge, the nodes can make fast, local decisions using CoAP communication. The Cloud application provides central monitoring and macro-level grid control.

The best feature of this project is its resilience. The stress tests proved that the system survives hardware faults and network losses. The Edge Forward Fill and Cloud Dynamic Thresholding mechanisms adapt to problems automatically.